

1. Archivos creados (estructura actualizada)

Se crearán los siguientes archivos según la arquitectura original:

```
...
el-ultimo-bastion/
├── main.py          # Punto de entrada del programa
├── config.py        # Constantes de configuración (resolución, FPS, etc.)
├── src/
│   └── core/
│       ├── __init__.py  # Inicializador del paquete (vacío)
│       ├── game.py      # Clase Game (bucle principal, gestión de estados)
│       └── states.py    # Máquina de estados (State, MenuState, GameplayState)
...
```

El resto de carpetas (`entities`, `managers`, `map`, `ui`, `utils`, `assets`, `docs`, `tests`, `data`) permanecen tal como se definieron, pero por ahora están vacías.

2. Código completo de cada archivo

2.1 `config.py`

```
```python
"""
Configuración global del videojuego "El Último Bastión".
Contiene constantes de ventana, rendimiento y colores básicos.
"""

Dimensiones de la ventana
SCREEN_WIDTH = 1024
SCREEN_HEIGHT = 768

FPS objetivo
FPS = 60

Título de la ventana
GAME_TITLE = "El Último Bastión"

Colores (RGB)
COLOR_BLACK = (0, 0, 0)
COLOR_WHITE = (255, 255, 255)
COLOR_RED = (255, 0, 0)
COLOR_GREEN = (0, 255, 0)
COLOR_BLUE = (0, 0, 255)
COLOR_GREY = (100, 100, 100)
COLOR_DARK_BG = (20, 20, 40) # Fondo oscuro para el menú
```
```

```
COLOR_MENU_TEXT = (220, 220, 220)
```

```
'''
```

```
### 2.2 `main.py`
```

```
```python
```

```
'''
```

Punto de entrada de "El Último Bastión".

Inicializa Pygame y lanza la aplicación a través de la clase Game.

```
'''
```

```
import pygame
```

```
import sys
```

```
from src.core.game import Game
```

```
def main():
```

```
 """Función principal que ejecuta el juego."""
```

```
 pygame.init()
```

```
 game = Game()
```

```
 game.run()
```

```
 pygame.quit()
```

```
 sys.exit()
```

```
if __name__ == "__main__":
```

```
 main()
```

```
'''
```

```
2.3 `src/core/__init__.py`
```

Archivo vacío que permite tratar el directorio `core` como paquete Python.

```
```python
```

```
# Paquete core del motor del juego
```

```
'''
```

```
### 2.4 `src/core/game.py`
```

```
```python
```

```
'''
```

Motor principal del juego.

Contiene la clase Game, que administra el bucle principal, los estados y el cambio entre pantallas.

```
'''
```

```
import pygame
```

```
import time
```

```
from config import SCREEN_WIDTH, SCREEN_HEIGHT, FPS, GAME_TITLE
```

```
from src.core.states import MenuState
```

```

class Game:
 """
 Clase principal del videojuego.
 Responsable de inicializar la ventana, ejecutar el bucle,
 medir el delta time y delegar en el estado actual.
 """

 def __init__(self):
 """Inicializa la ventana, el reloj y el estado inicial."""
 self.screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
 pygame.display.set_caption(GAME_TITLE)
 self.clock = pygame.time.Clock()
 self.running = False
 self.state = None

 # Referencia al estado actual (se establece en run())
 self.change_state(MenuState(self))

 def change_state(self, new_state):
 """
 Cambia el estado actual de la máquina de estados.
 Args:
 new_state: instancia de una subclase de State.
 """
 self.state = new_state

 def handle_events(self):
 """
 Procesa los eventos de Pygame.
 Cada estado puede tener su propio manejo, pero eventos
 globales como QUIT se tratan aquí.
 """
 for event in pygame.event.get():
 if event.type == pygame.QUIT:
 self.running = False
 else:
 # Delegar el resto de eventos al estado activo
 self.state.handle_events(event)

 def update(self, dt):
 """
 Actualiza la lógica del estado actual.
 Args:
 dt: delta time en segundos desde el último frame.
 """
 self.state.update(dt)

```

```

def render(self):
 """
 Renderiza la pantalla actual.
 Limpia la pantalla y delega en el estado activo.
 """
 self.screen.fill((0, 0, 0)) # Limpieza global (el estado puede sobrescribir)
 self.state.render(self.screen)
 pygame.display.flip()

def run(self):
 """Bucle principal del juego."""
 self.running = True
 last_time = time.time()

 while self.running:
 # Calcular delta time (segundos)
 current_time = time.time()
 dt = current_time - last_time
 last_time = current_time

 self.handle_events()
 self.update(dt)
 self.render()
 self.clock.tick(FPS) # Limitar los FPS
 ...

2.5 `src/core/states.py`

```python
"""
Máquina de estados del juego.
Define la clase base State y las implementaciones concretas
para el menú principal y la fase de juego (placeholder).
"""

import pygame
from config import COLOR_DARK_BG, COLOR_MENU_TEXT, COLOR_WHITE,
COLOR_BLUE
import sys

class State:
    """
    Clase base abstracta para todos los estados del juego.
    Define la interfaz común que deben implementar los estados concretos.
    """
    def __init__(self, game):
        """
        Args:

```

```

        game: referencia a la instancia principal de Game.
        """
        self.game = game

    def handle_events(self, event):
        """Procesa un evento de Pygame."""
        raise NotImplementedError

    def update(self, dt):
        """Actualiza la lógica del estado. dt en segundos."""
        raise NotImplementedError

    def render(self, screen):
        """Dibuja el estado en la superficie dada."""
        raise NotImplementedError


class MenuState(State):
    """
    Pantalla de menú principal.
    Muestra las opciones 'Jugar' y 'Salir'.
    """
    def __init__(self, game):
        super().__init__(game)
        # Fuente para el texto
        self.title_font = pygame.font.Font(None, 72)
        self.option_font = pygame.font.Font(None, 48)

        # Textos del menú
        self.title_text = self.title_font.render("EL ÚLTIMO BASTIÓN", True,
COLOR_MENU_TEXT)
        self.play_text = self.option_font.render("Presiona ENTER para Jugar", True,
COLOR_WHITE)
        self.quit_text = self.option_font.render("Presiona ESC para Salir", True,
COLOR_WHITE)

        # Posiciones
        screen_center_x = self.game.screen.get_width() // 2
        screen_center_y = self.game.screen.get_height() // 2

        self.title_rect = self.title_text.get_rect(center=(screen_center_x, screen_center_y - 100))
        self.play_rect = self.play_text.get_rect(center=(screen_center_x, screen_center_y +
20))
        self.quit_rect = self.quit_text.get_rect(center=(screen_center_x, screen_center_y + 80))

    def handle_events(self, event):
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN:

```

```

        # Iniciar el juego (por ahora un placeholder)
        from src.core.states import GameplayState
        self.game.change_state(GameplayState(self.game))
    elif event.key == pygame.K_ESCAPE:
        self.game.running = False # Salir del juego

def update(self, dt):
    # El menú no necesita actualización por ahora
    pass

def render(self, screen):
    # Fondo oscuro
    screen.fill(COLOR_DARK_BG)
    # Dibujar textos
    screen.blit(self.title_text, self.title_rect)
    screen.blit(self.play_text, self.play_rect)
    screen.blit(self.quit_text, self.quit_rect)

class GameplayState(State):
    """
    Estado placeholder que representa la fase de juego.
    En esta versión solo muestra un fondo azul e indica que el juego
    está activo. Más adelante contendrá la lógica completa de oleadas, defensas, etc.
    """
    def __init__(self, game):
        super().__init__(game)
        self.font = pygame.font.Font(None, 36)
        self.info_text = self.font.render("Juego en desarrollo... (ESC para volver)", True,
COLOR_WHITE)
        self.info_rect = self.info_text.get_rect(center=(self.game.screen.get_width()/2,
self.game.screen.get_height()/2))

    def handle_events(self, event):
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_ESCAPE:
                # Regresar al menú principal
                self.game.change_state(MenuState(self.game))

    def update(self, dt):
        # En el futuro actualizará enemigos, defensas, oleadas, etc.
        pass

    def render(self, screen):
        # Fondo azul sencillo
        screen.fill(COLOR_BLUE)
        screen.blit(self.info_text, self.info_rect)
...

```

3. Explicación técnica detallada

`config.py`

Centraliza las constantes globales: tamaño de ventana (1024×768, resolución clásica para PC), FPS objetivo (60) y una paleta de colores básica. Esta separación evita números mágicos y facilita ajustes futuros.

`main.py`

- Importa `pygame` y la clase `Game`.
- Llama a `pygame.init()` para inicializar todos los módulos de Pygame.
- Crea una instancia de `Game` y ejecuta su método `run()`.
- Al finalizar, limpia con `pygame.quit()` y sale con `sys.exit()`.
- Se ajusta al estándar `if \_\_name\_\_ == "\_\_main\_\_":`, permitiendo importar el módulo sin ejecutar.

`src/core/game.py` – Clase `Game`

- **Constructor**: crea la ventana con las dimensiones de `config.py`, establece el título, inicializa `pygame.time.Clock` y asigna el estado inicial (`MenuState`).
- **change_state(new_state)**: método que permite transiciones entre pantallas. Simplemente guarda la referencia al nuevo estado.
- **handle_events()**: itera sobre todos los eventos de Pygame. Si el evento es `QUIT` (cerrar ventana), detiene el bucle. Los demás eventos se pasan al estado actual mediante `self.state.handle\_events(event)`.
- **update(dt)**: llama al método `update` del estado activo con el tiempo transcurrido (dt). En esta versión, ni `MenuState` ni `GameplayState` utilizan el dt, pero está preparado para futuras lógicas de movimiento/animación.
- **render()**: pinta un fondo negro general (por si algún estado no cubre toda la pantalla), delega el dibujo específico en el estado y finalmente actualiza la pantalla con `pygame.display.flip()`.
- **run()**: bucle principal. Calcula el delta time con `time.time()` (precisión suficiente para la mayoría de juegos 2D), llama a los métodos de procesamiento y limita los FPS con `clock.tick(FPS)`. La variable `running` controla la ejecución.

`src/core/states.py` – Sistema de estados

- **State** (clase base): define los métodos abstractos `handle\_events`, `update` y `render`. Todos reciben la referencia a `Game` para acceder a la pantalla, cambiar de estado o modificar `running`.
- **MenuState**:
 - Prepara fuentes y superficies con texto (título, “Jugar”, “Salir”).
 - En `handle\_events`, captura `K\_RETURN` (Enter) para cambiar al estado `GameplayState`, y `K\_ESCAPE` para salir (`game.running = False`).
 - `render` dibuja el fondo oscuro y los tres textos centrados.
- **GameplayState** (placeholder):
 - Muestra un fondo azul y un texto “Juego en desarrollo... (ESC para volver)”.
 - En `handle\_events`, permite volver al menú con `K\_ESCAPE`.

- Futuramente alojará los managers de oleadas, enemigos, torres, etc.

4. Instrucciones de ejecución

1. **Requisitos previos**:

- Python 3.7 o superior.
- Pygame instalado (`pip install pygame`).

2. **Estructura de carpetas**:

Colocar todos los archivos exactamente con la jerarquía indicada:

...

el-ultimo-bastion/

```
|— main.py
|— config.py
|— src/
    |— core/
        |— __init__.py
        |— game.py
        |— states.py
```

...

3. **Ejecutar el juego**:

- Abrir una terminal en la carpeta raíz `el-ultimo-bastion`.
- Ejecutar: `python main.py`

4. **Controles durante la ejecución**:

- **Menú principal**:
 - `ENTER`: comienza la partida (estado juego placeholder).
 - `ESC`: sale del juego.
- **Pantalla de juego (placeholder)**:
 - `ESC`: regresa al menú principal.
- **Cerrar ventana**: el botón de cerrar de la ventana también finaliza el programa.

5. Flujo de ejecución del programa

1. `main.py` → `pygame.init()` → `Game()` → se crea ventana, reloj, estado inicial `MenuState`.
2. `game.run()` inicia el bucle:
 - **Eventos**: si `QUIT` → `running = False`. Si no, se pasan a `MenuState.handle\_events()`.
 - **Actualización**: `MenuState.update(dt)` (vacío por ahora).
 - **Renderizado**: pantalla negra base + `MenuState.render()` (fondo oscuro, textos).
 - `clock.tick(60)` mantiene los FPS.
3. Cuando el usuario presiona **ENTER** en el menú:

- `MenuState.handle_events()` crea una instancia de `GameplayState` y llama a `game.change_state()`.
 - Ahora el estado activo es `GameplayState`.
4. En `GameplayState`:
 - Se muestran fondo azul y texto.
 - Al presionar **ESC**, se vuelve a crear `MenuState` y se cambia.
 5. El bucle continúa hasta que `running` sea `False` (por cierre de ventana o por `ESC` en el menú).
 6. `pygame.quit()` y `sys.exit()` finalizan limpiamente.

Este esqueleto funcional sienta las bases de la arquitectura y permite añadir progresivamente el resto de módulos (enemigos, defensas, oleadas) sin modificar la estructura central, cumpliendo con los principios de modularidad y escalabilidad definidos en el diseño original.